UFRN
UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE

Processador Programável

Circuitos Digitais

Prof. Dr. Samuel Xavier

Grupo

Carlos Eduardo Nascimento Morais
Gisliardi Araújo Santos
Juan Pablo Hortêncio Ferreira
Pedro Henrique da Silva Santos
Pedro Henrique Leite dos Santos

Introdução

Unificação de todos os conteúdos aprendidos durante as aulas

- Tabela verdade
- Máquina de Estado Finito (FSM)
- Datapaths
- Projeto RTL e Máquina de Estado de Alto Nível (HLSM)
- Processador programável de 6 instruções

Extensão do Processador programável de 6 instruções para lidar com uma sétima instrução - O JLT (Jump If Less Than)

Definição do projeto

Essa nova instrução introduz um desvio condicional que verifica se o valor de um registrador é menor que o de outro e, se for, será realizado um salto de instruções

Instrução	Significado	Opcode
MOV Ra, d	RF[a] = D[d]	0000
MOV d, Ra	D[d] = RF[a]	0001
ADD Ra, Rb, Rc	RF[a] = RF[b] + RF[c]	0010
MOV Ra, #C	RF[a] = C	0011
SUB Ra, Rb, Rc	RF[a] = RF[b] - RF[c]	0100
JMPZ Ra, offset	PC = PC + offset if RF[a] = 0	0101
JLT Ra, Rb, Rc	PC = PC + 2, if RF[a] < RF[b]	0110

Desenvolvimento

Relembrando o Projeto RTL...

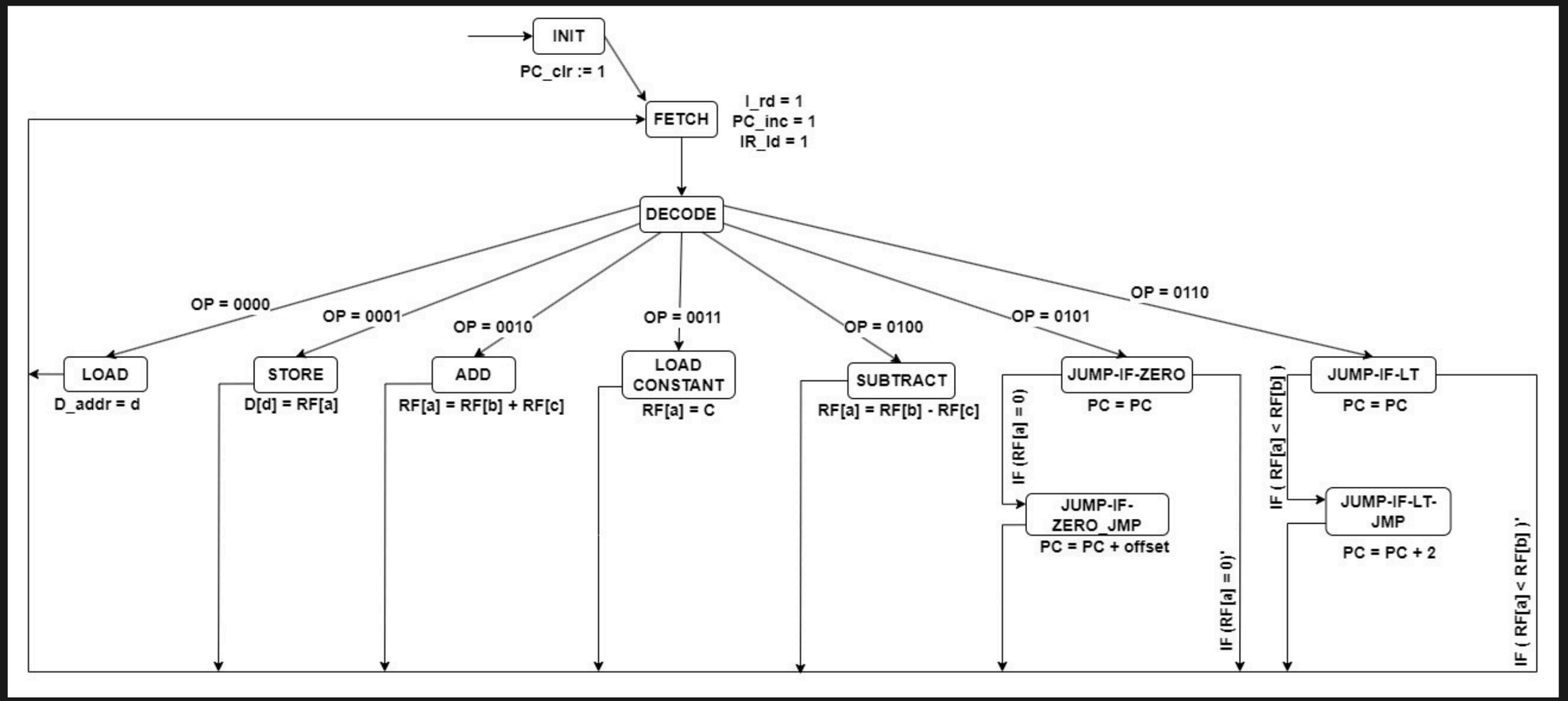
1. Capturar o funcionamento do circuito usando uma máquina de estados finitos de alto nível (HLSM);
2. Projetar o Datapath;
3. Conectar o Datapath ao controlador;
4. Derivar o controlador da máquina de estados finitos.

Desenvolvimento - Step 1

HLSM

Objetivo

Descrever todo o comportamento do processador e descrever as transições



Desenvolvimento – Step 2 e 3

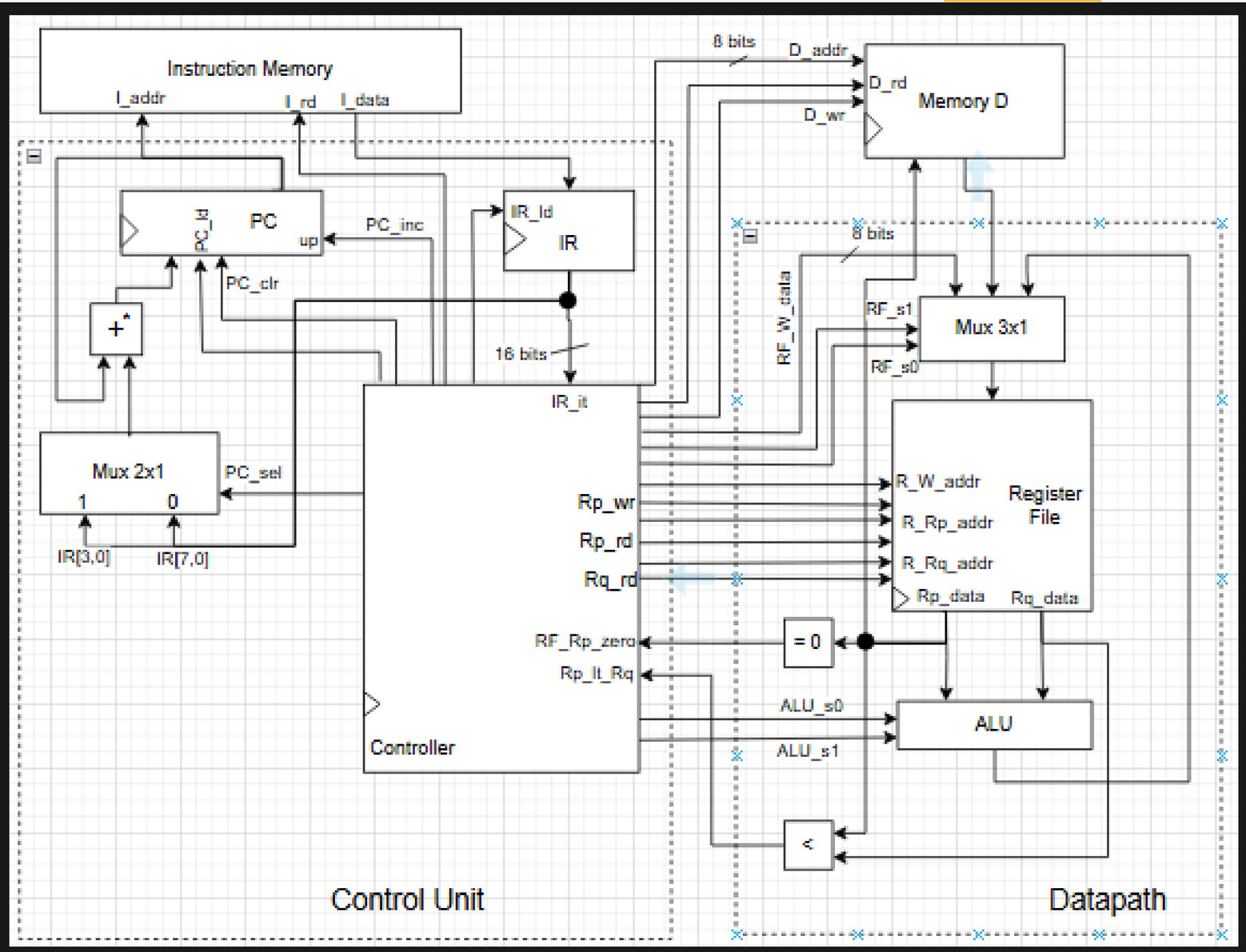
Datapath e Unidade de controle

Objetivo

Descrever o caminho de dados do processador

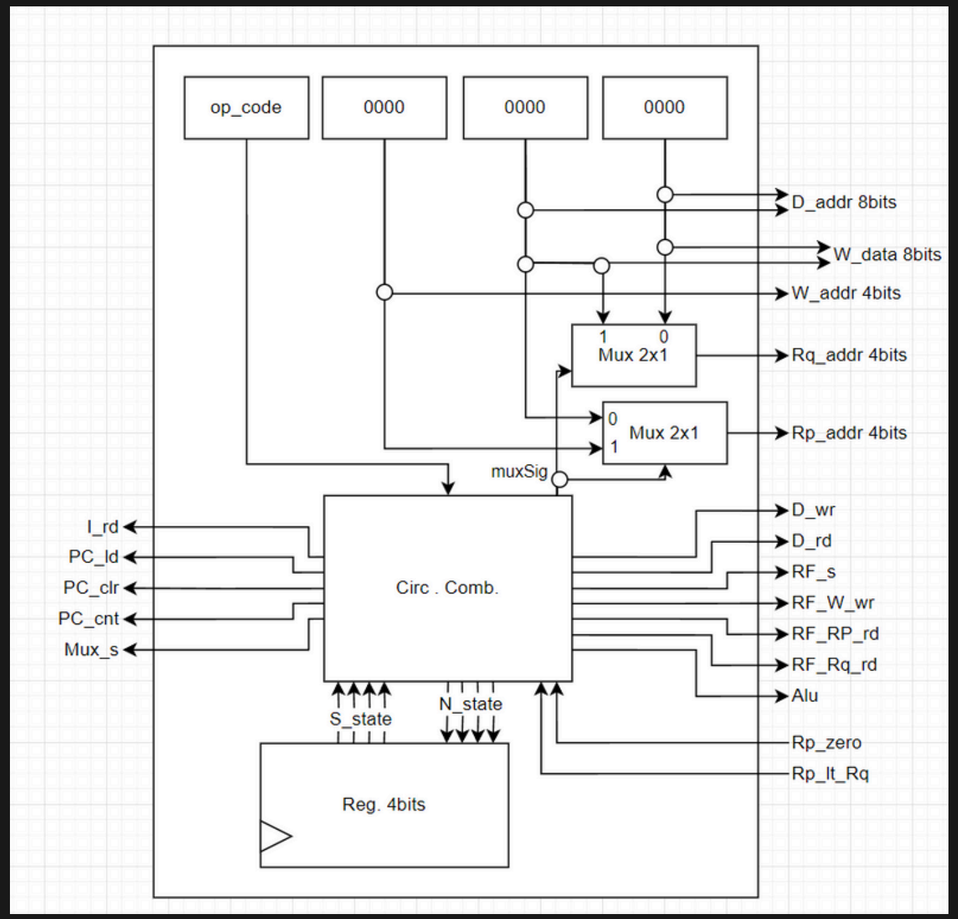
Pontos importantes

- Incremento dinâmico do PC
- Uso do MUX 3x1 para lidar com o dado que será carregado no Register File



Desenvolvimento – Step 2 e 3

Controller

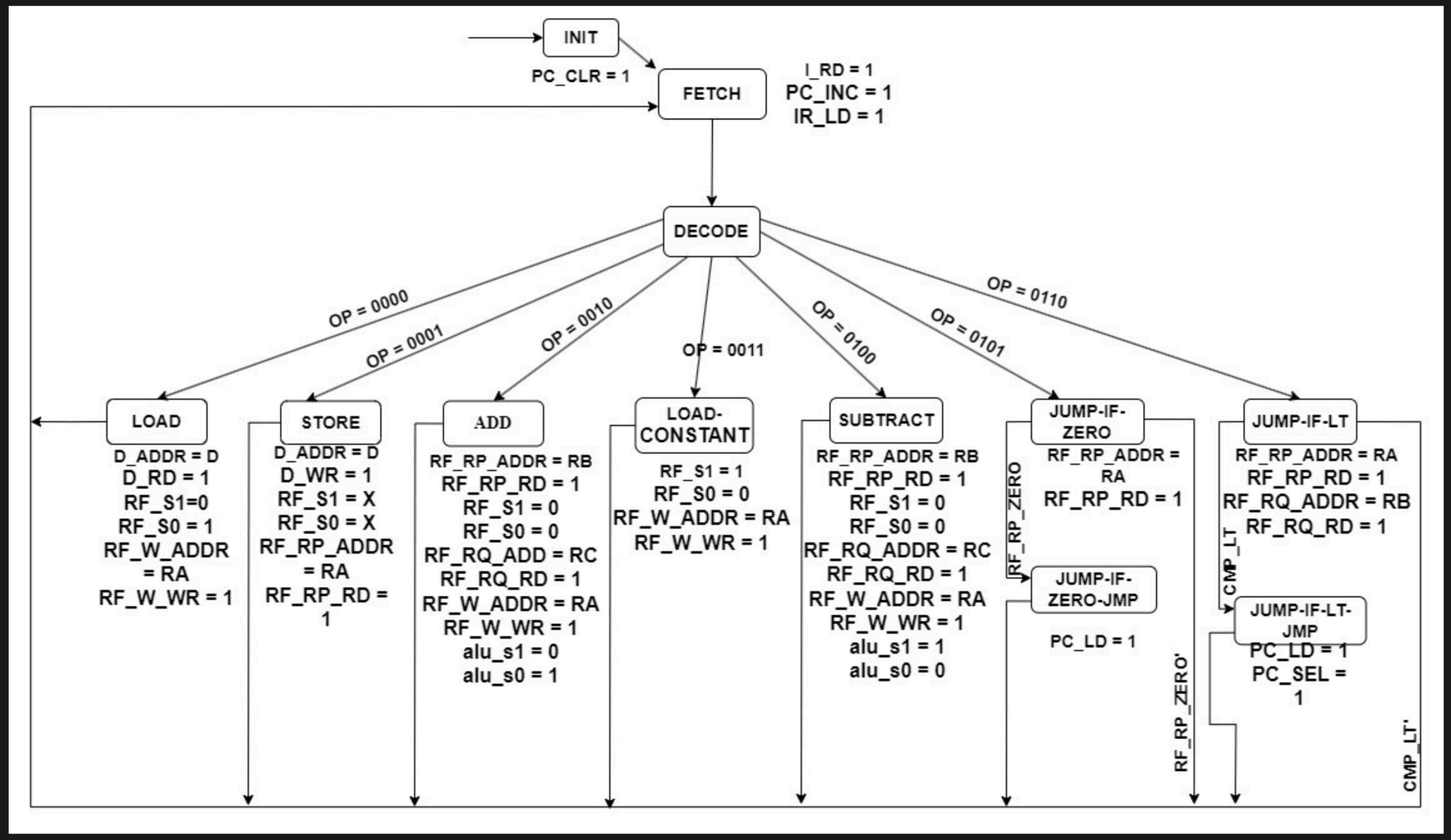


Pontos importantes

- Circuito combinacional para guiar o caminho de dados

Desenvolvimento - Step 4

Derivar a FSM



Desenvolvimento - Step 4

Desenvolvimento da tabela verdade a partir da FSM

	S(3)	S(2)	S(1)	S(0)	RP_z	RP_lt_RQ	X	Y	Z	N3	N2	N1	N0	D_wr	D_rd	mux	W_wr	Rp_rd	Rq_rd	alu	muxSig	I_rd	PC_ld	PC_mux	PC_cnt	PC_clr	IR_ld
init	0	0	0	0	*	*	*	*	*	0	0	0	1													1	
fetch	0	0	0	1	*	*	*	*	*	0	0	1	0									1			1		1
decode	0	0	1	0	*	*	0	0	0	0	0	1	1														
	0	0	1	0	*	*	0	0	1	0	1	0	0														
	0	0	1	0	*	*	0	1	0	0	1	0	1														
	0	0	1	0	*	*	0	1	1	0	1	1	0														
	0	0	1	0	*	*	1	0	0	0	1	1	1														
	0	0	1	0	*	*	1	0	1	1	0	0	0														
	0	0	1	0	*	*	1	1	1	1	0	1	0														
load	0	0	1	1	*	*	*	*	*	0	0	0	1		1	01	1										
store	0	1	0	0	*	*	*	*	*	0	0	0	1	1				1									
add	0	1	0	1	*	*	*	*	*	0	0	0	1				1	1	1	01	1						
constant	0	1	1	0	*	*	*	*	*	0	0	0	1			10	1										
subtract	0	1	1	1	*	*	*	*	*	0	0	0	1				1	1	1	10	1						
Jump if zero	1	0	0	0	0	*	*	*	*	0	0	0	1					1	1								
					1	*	*	*	*	1	0	0	1														
jmp if zero jmp	1	0	0	1	*	*	*	*	*	0	0	0	1										1				
jump if lt	1	0	1	0	*	0	*	*	*	0	0	0	1					1	1								
					*	1	*	*	*	1	0	1	1														
jmp if lt jmp	1	0	1	1	*	*	*	*	*	0	0	0	1										1		1		

Desenvolvimento - Step 4

Demonstração da estrutura de passagem dos estados

```
--STATES
INIT   <= not S(3) and not S(2) and not S(1) and not S(0);
FETCH  <= not S(3) and not S(2) and not S(1) and     S(0);
DECODE <= not S(3) and not S(2) and     S(1) and not S(0);
LOAD   <= not S(3) and not S(2) and     S(1) and     S(0);
STORE  <= not S(3) and     S(2) and not S(1) and not S(0);
ADD      <= not S(3) and     S(2) and not S(1) and     S(0);
CONST  <= not S(3) and     S(2) and     S(1) and not S(0);
SUB    <= not S(3) and     S(2) and     S(1) and     S(0);
JMPZ   <=     S(3) and not S(2) and not S(1) and not S(0);
JMPZJ  <=     S(3) and not S(2) and not S(1) and     S(0);
JMPLT  <=     S(3) and not S(2) and not S(1) and     S(0);
JMPLTJ <=     S(3) and not S(2) and not S(1) and not S(0);

--OPCODE
toLOAD  <= not IR_it(14) and not IR_it(13) and not IR_it(12);
toSTORE <= not IR_it(14) and not IR_it(13) and     IR_it(12);
toADD   <= not IR_it(14) and     IR_it(13) and not IR_it(12);
toCONST <= not IR_it(14) and     IR_it(13) and     IR_it(12);
toSUB   <=     IR_it(14) and not IR_it(13) and not IR_it(12);
toJMPZ  <=     IR_it(14) and not IR_it(13) and     IR_it(12);
toJMPLT <=     IR_it(14) and     IR_it(13) and     IR_it(12);
```

```
--NEXT STATE
N(3) <= (JMPZ and RP_zero) or
        (JMPLT and RP_lt_RQ) or
        (DECODE and (toJMPZ or toJMPLT));

N(2) <= (DECODE and (toSTORE or toADD or toCONST or toSUB));

N(1) <= FETCH or
         (JMPLT and RP_lt_RQ) or
        (DECODE and (toADD or toCONST or toSUB or toJMPLT));

N(0) <= INIT or
         not(FETCH or (DECODE and (toSTORE or toCONST or toJMPLT or toJMPLT)));
```

Demonstração!

Bloco de carregamento

CONST ← RF(0) = 0
CONST ← RF(1) = 1
CONST ← RF(2) = 0
CONST ← RF(3) = 8

Bloco de inicialização

STORE ← D(0) = RF(3)

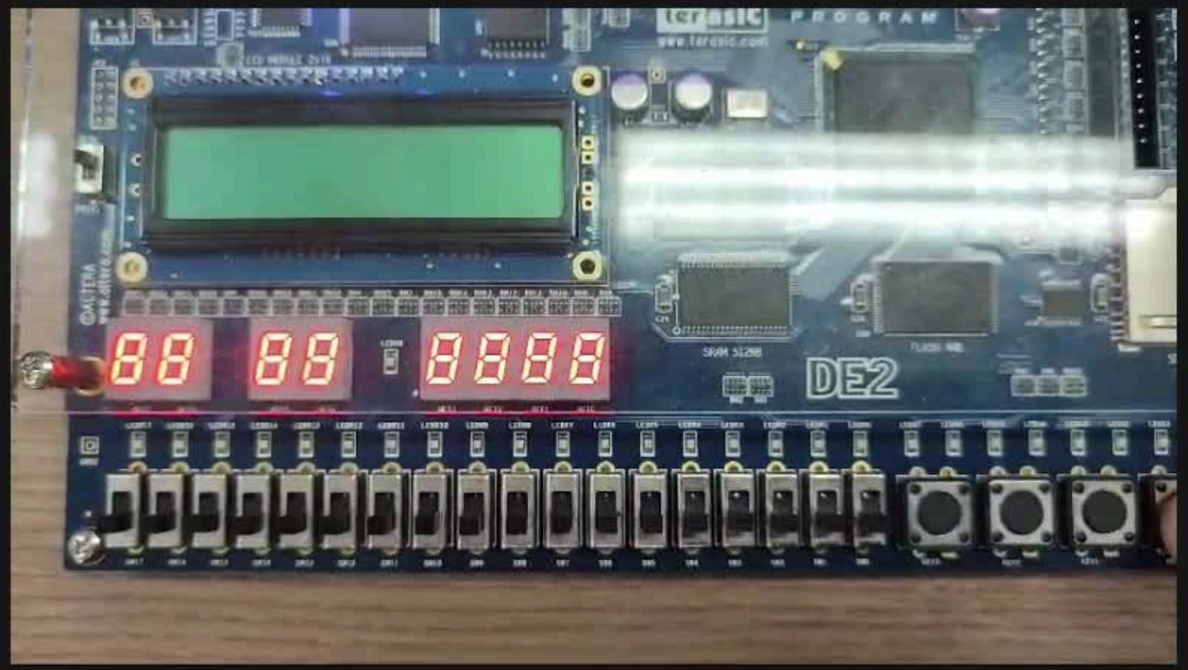
Loop principal

STORE ← D(0) = RF(2)
ADD ← RF(2) = RF(2) + RF(1)
JMPLT ← RF(3) < RF(2), JUMP 2
JMPZ ← RF(0) = 0, JUMP -3

Bloco final

CONST ← RF(2) = 255
STORE ← D(0) = RF(2)

Demonstração!



Bloco de carregamento

CONST ← RF(0) = 0
CONST ← RF(1) = 1
CONST ← RF(2) = 0
CONST ← RF(3) = 8



Bloco de inicialização

STORE ← D(0) = RF(3)

Loop principal

STORE ← D(0) = RF(2)
ADD ← RF(2) = RF(2) + RF(1)
JMPLT ← RF(3) < RF(2), JUMP 2
JMPZ ← RF(0) = 0, JUMP -3

Bloco final

CONST ← RF(2) = 255
STORE ← D(0) = RF(2)

Dificuldades

1. Lidar com a quantidade de informações na HLSM;
2. Fazer o diagrama da unidade de controle e datapath com os componentes necessários para a extensão.
3. Codificar o processador.

Obrigado!

